

INFB6074 - INFRAESTRUCTURA PARA CS. DE DATOS
INGENIERÍA CIVIL EN CIENCIA DE DATOS

Etapas final del Proyecto

“Desarrollo del Proyecto y Presentación Final”

Dr. Ing. Michael Miranda Sandoval

Mayo de 2025

ETAPA FINAL DEL PROYECTO

INTEGRANTES:

Nombre: Felipe Martínez González

Email: fmartinezgo@utem.cl

Nombre: Christian Pérez Flores

Email: cperezfl@utem.cl

Nombre: Benjamín Zamorano Soto

Email: bzamoranos@utem.cl

Introducción

El crecimiento de los desafíos agrícolas y el volumen masivo de datos no estructurados en forma de imágenes han motivado el uso de infraestructura avanzada para resolver problemas de diagnóstico vegetal. El proyecto **PlagaZero** se posiciona como una herramienta de detección temprana de enfermedades en cultivos mediante imágenes digitales, integrando modelos de aprendizaje profundo y desplegándose mediante tecnologías modernas como **Docker** y **Streamlit**.

Este informe se enfoca en el diseño, implementación y despliegue de la infraestructura del sistema, así como en el uso de técnicas de computación paralela y distribución, esenciales para el entrenamiento y operación eficiente sobre un conjunto de datos de gran escala.

Datos y entorno de entrenamiento

El modelo fue entrenado utilizando un conjunto de aproximadamente **20.000 imágenes**, lo cual constituye un volumen significativo para un sistema personal. Estas imágenes representan **datos no estructurados**, cuya manipulación y procesamiento requiere infraestructura adaptada a flujos de trabajo intensivos en cómputo.

El equipo utilizado para el entrenamiento cuenta con las siguientes características:

- **Procesador:** AMD Ryzen 5 5600X (6 núcleos, 12 hilos, 3.70 GHz)
- **Memoria RAM:** 32 GB DDR4
- **Nombre del equipo:** DESKTOP-4074RG9

Este hardware permitió ejecutar entrenamientos en paralelo a través de múltiples hilos de CPU, aprovechando la arquitectura multihilo del procesador para acelerar operaciones de lectura de datos, preprocesamiento y entrenamiento.

Computación paralela y Big Data

Computación Paralela

Durante el entrenamiento, se aplicaron técnicas de paralelización de datos a nivel de CPU:

- **Paralelización en carga de datos:** Utilizando `torch.utils.data.DataLoader` con múltiples `workers`, se logró leer y transformar imágenes en paralelo desde disco.
- **Batch training:** El modelo U-Net fue entrenado utilizando procesamiento por lotes, permitiendo calcular gradientes de múltiples imágenes de forma simultánea.
- **Vectorización:** Operaciones con matrices fueron optimizadas usando `NumPy` y `PyTorch`, lo cual permite aprovechar instrucciones SIMD a nivel de CPU.



UNIVERSIDAD
TECNOLÓGICA
METROPOLITANA

Enfoque Big Data

Si bien el sistema fue desarrollado en un entorno local, el volumen de imágenes (20.000) justifica un enfoque de ciencia de datos a gran escala. Se aplicaron principios de Big Data en:

- **Almacenamiento eficiente:** Uso de formatos comprimidos (.jpg/.png) y acceso por lotes.
- **Preprocesamiento distribuido:** Separación en etapas de segmentación y clasificación, cada una trabajando con diferentes subconjuntos de datos.
- **Pipeline modular:** Arquitectura desacoplada permite escalar fácilmente hacia entornos distribuidos como Spark o clusters en la nube.

Diseño de la arquitectura propuesta

El sistema se compone de tres capas principales:

- **Frontend:** Streamlit como plataforma visual para subir imágenes y ver resultados.
- **Backend:** Modelos U-Net (PyTorch) para segmentación y CNN (TensorFlow) para clasificación.
- **Contenedores:** Docker como entorno de ejecución unificado y replicable.

Cada componente puede ser desacoplado, lo que habilita una futura integración con sistemas distribuidos o en la nube, como Kubernetes o servicios de AWS/GCP.

Implementación y despliegue

La solución fue empacada en un contenedor Docker que incluye todas las dependencias necesarias. Se utilizó una imagen base ligera de Python 3.11 y un archivo `requirements.txt` con librerías de ciencia de datos, visión por computador y frameworks de deep learning.

El despliegue se realiza localmente mediante:

```
docker build -t plagazero_app .  
docker run -p 8501:8501 plagazero_app
```

El contenedor expone un servidor web accesible desde el navegador en `localhost:8501`, facilitando la interacción incluso desde equipos remotos si se habilita la red.

Resultados y discusión

El sistema entregó resultados visuales de alta calidad en tiempos aceptables considerando que fue ejecutado en CPU. La arquitectura modular y contenedorizada permite escalar fácilmente el procesamiento si se dispone de GPU o clusters.

La separación de tareas (clasificación vs. segmentación) permitió aplicar estrategias específicas a cada fase, y la infraestructura elegida (Docker + Streamlit) demostró ser adecuada tanto para pruebas como para despliegue final.

Conclusiones

PlagaZero es un caso exitoso de aplicación de técnicas de ciencia de datos e infraestructura moderna a un problema del mundo real. El uso de contenedores permitió encapsular un sistema complejo con múltiples dependencias, mientras que la adopción de prácticas de computación paralela y procesamiento eficiente habilitó el trabajo con grandes volúmenes de imágenes.

Para escenarios futuros, se contempla migrar a plataformas distribuidas (como Apache Spark) o servidores con soporte GPU, junto con la integración de pipelines automatizados para entrenamiento y despliegue continuo.

1. Referencias

1. PlantVillage Dataset. (2018). Kaggle. <https://www.kaggle.com/datasets/emmarex/plantdisease>
2. Chat GPT. (2022). <https://openai.com/es-ES/index/chatgpt/>

Palabras clave: Plagas agrícolas, Segmentación, Clasificación, Streamlit, Docker, Ciencia de Datos.